WEEK 4: TRAVEL BOOKING MANAGEMENT SYSTEM

Scenario

The Travel Booking Management System is designed to manage customer bookings for different travel packages. The system stores information about customers, their bookings, and the individual travel packages included in each booking.

The Booking table stores the main information about a booking, such as the customer who made the booking, booking date, total amount, and booking status.

The BookingDetails table stores the details of the travel packages included in each booking, such as the package ID, number of travellers, and package price.

This system helps maintain booking information accurately and establishes relationships between customers, bookings, and travel packages.

Design Booking and BookingDetails Tables

Booking Table

The Booking table stores the main information related to customer travel bookings.

Attributes:

- **Booking_ID** – Unique identifier for each booking.
- **Customer_ID** – Identifies the customer who made the booking.
- **Booking_Date** – Stores the date on which the booking was made.
- **Total_Amount** – Stores the total cost of the booking.
- **Booking_Status** – Stores the current status of the booking, such as Confirmed, Processing, or Completed.

BookingDetails Table

The BookingDetails table stores information about the individual travel packages included in a booking.

Attributes:

- **BOOKING_DETAILS_ID** – Unique identifier for each booking detail.
- **Booking_ID** – Identifies the corresponding booking.
- **Package_ID** – Identifies the travel package selected.
- **Number_of_Travellers** – Stores the number of people travelling.
- **Price** – Stores the price of the selected travel package.

```
SQL> create table booking(booking_id varchar(10),customer_id number, booking_date date, total_amount number,booking_status varchar(20));
Table created.
SQL> create table bookingdetails(booking_details_id varchar2(10),booking_id varchar2(10),package_id number,number_of_travellers number,price number);
Table created.
```

Establish Relationships

Customer and Booking

One customer can make multiple bookings, while each booking belongs to one customer.

Relationship:

Customer → Booking = One-to-Many

The Customer_ID in the Booking table establishes this relationship.

Booking and BookingDetails

One booking can contain multiple booking details, while each booking detail belongs to one booking.

Relationship:

Booking → BookingDetails = One-to-Many

The Booking_ID in BookingDetails connects each booking detail with its corresponding booking.

TravelPackage and BookingDetails

The BookingDetails table is connected to the TravelPackage table through Package_ID.

This identifies the travel package selected by the customer.

Relationship:

TravelPackage → BookingDetails

This connects the available travel packages with customer booking information.

These relationships follow the same structure as the Customer → Orders, Orders → OrderDetails, and product → OrderDetails relationships in the source.

4. Apply Constraints

Constraints are used to maintain the accuracy and integrity of the database.

Primary Key

Primary keys uniquely identify records.

- Booking_ID → Primary Key of Booking
- BOOKING_DETAILS_ID → Primary Key of BookingDetails

Foreign Key

Foreign keys establish relationships between tables.

- Customer_ID in Booking references Customer.
- Booking_ID in BookingDetails references Booking.
- Package_ID in BookingDetails references TravelPackage.

NOT NULL Constraint

NOT NULL ensures that important information cannot be left empty.

In our system, important fields such as:

- Customer ID
- Booking Date
- Booking Status
- Number of Travellers
- Price

are required.

CHECK Constraint

CHECK constraints ensure that only valid values are entered.

Our system uses:

- Number of Travellers must be greater than 0.
- Price must be greater than 0.
- Total Amount must be greater than or equal to 0.

These correspond to the source's requirements for positive quantity, positive price, and non-negative total amount.

PRIMARY KEY CONSTRAINT:

```
SQL> ALTER TABLE Booking
  2  ADD CONSTRAINT PK_BOOKING
  3  PRIMARY KEY (Booking_ID);

Table altered.
```

```
SQL> ALTER TABLE BookingDetails
  2  ADD CONSTRAINT PK_BOOKINGDETAILS
  3  PRIMARY KEY (BOOKING_DETAILS_ID);

Table altered.
```

FOREIGN KEY CONSTRAINT:

```
SQL> ALTER TABLE Booking
  2  ADD CONSTRAINT FK_BOOKING_CUSTOMER
  3  FOREIGN KEY (Customer_ID)
  4  REFERENCES Customer(Customer_ID);

Table altered.
```

```
Table altered.

SQL> ALTER TABLE BookingDetails
  2  ADD CONSTRAINT FK_BOOKING_DETAILS
  3  FOREIGN KEY (Booking_ID)
  4  REFERENCES Booking(Booking_ID);

Table altered.
```

```
SQL> ALTER TABLE BookingDetails
  2  ADD CONSTRAINT FK_BOOKING_DETAILS
  3  FOREIGN KEY (Booking_ID)
  4  REFERENCES Booking(Booking_ID);

Table altered.
```

NOT NULL CONSTRAINT:

```
SQL> ALTER TABLE Booking
  2  MODIFY Customer_ID NUMBER NOT NULL;

Table altered.

SQL> ALTER TABLE Booking
  2  MODIFY Booking_Date DATE NOT NULL;

Table altered.

SQL> ALTER TABLE Booking
  2  MODIFY Booking_Status VARCHAR2(20) NOT NULL;

Table altered.

SQL> ALTER TABLE BookingDetails
  2  MODIFY Number_of_Travellers NUMBER NOT NULL;

Table altered.

SQL> ALTER TABLE BookingDetails
  2  MODIFY Price NUMBER NOT NULL;

Table altered.
```

CHECK CONSTRAINT:

```
SQL> ALTER TABLE BookingDetails
  2  ADD CONSTRAINT CHK_TRAVELLERS
  3  CHECK (Number_of_Travellers > 0);

Table altered.
```

```
SQL> ALTER TABLE BookingDetails
  2  ADD CONSTRAINT CHK_BOOKING_PRICE
  3  CHECK (Price > 0);

Table altered.
```

```
SQL> ALTER TABLE Booking
  2  ADD CONSTRAINT CHK_TOTAL_AMOUNT
  3  CHECK (Total_Amount >= 0);

Table altered.
```

DESCRIBE THE STRUCTURE OF THE TABLES:

```
SQL> DESC Booking;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------
----
 BOOKING_ID                                NOT NULL VARCHAR2(10)
 CUSTOMER_ID                               NOT NULL NUMBER
 BOOKING_DATE                              NOT NULL DATE
 TOTAL_AMOUNT                                       NUMBER
 BOOKING_STATUS                            NOT NULL VARCHAR2(20)

SQL> DESC BookingDetails;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------
----
 BOOKING_DETAILS_ID                        NOT NULL VARCHAR2(10)
 BOOKING_ID                                         VARCHAR2(10)
 PACKAGE_ID                                         NUMBER
 NUMBER_OF_TRAVELLERS                      NOT NULL NUMBER
 PRICE                                     NOT NULL NUMBER
```

Insert Values

Sample records are inserted into the Booking and BookingDetails tables.

The Booking table contains information such as:

- Booking ID
- Customer ID
- Booking date
- Total amount
- Booking status

The BookingDetails table contains:

- Booking detail ID
- Booking ID
- Package ID

- Number of travellers
- Price

These sample values are useful for testing relationships, joins, reports, and CRUD operations.

```
SQL> INSERT INTO Booking
  2  VALUES ('B1', 301, TO_DATE('10-09-2026','DD-MM-YYYY'), 24000, 'Confirmed');

1 row created.

SQL>
SQL> INSERT INTO Booking
  2  VALUES ('B2', 302, TO_DATE('11-09-2026','DD-MM-YYYY'), 36000, 'Confirmed');

1 row created.

SQL>
SQL> INSERT INTO Booking
  2  VALUES ('B3', 303, TO_DATE('12-09-2026','DD-MM-YYYY'), 55000, 'Processing');

1 row created.

SQL>
SQL> INSERT INTO Booking
  2  VALUES ('B4', 304, TO_DATE('13-09-2026','DD-MM-YYYY'), 65000, 'Confirmed');

1 row created.

SQL>
SQL> INSERT INTO Booking
  2  VALUES ('B5', 305, TO_DATE('14-09-2026','DD-MM-YYYY'), 25000, 'Completed');

1 row created.

SQL>
SQL> INSERT INTO Booking
  2  VALUES ('B6', 306, TO_DATE('15-09-2026','DD-MM-YYYY'), 30000, 'Processing');

1 row created.

SQL>
SQL> INSERT INTO Booking
  2  VALUES ('B7', 307, TO_DATE('16-09-2026','DD-MM-YYYY'), 45000, 'Completed');

1 row created.
```

```
SQL> INSERT INTO BookingDetails
  2  VALUES ('BD1', 'B1', 101, 2, 12000);

1 row created.

SQL>
SQL> INSERT INTO BookingDetails
  2  VALUES ('BD2', 'B2', 102, 2, 18000);

1 row created.

SQL>
SQL> INSERT INTO BookingDetails
  2  VALUES ('BD3', 'B3', 103, 1, 55000);

1 row created.

SQL>
SQL> INSERT INTO BookingDetails
  2  VALUES ('BD4', 'B4', 104, 1, 65000);

1 row created.

SQL>
SQL> INSERT INTO BookingDetails
  2  VALUES ('BD5', 'B5', 105, 1, 25000);

1 row created.

SQL>
SQL> INSERT INTO BookingDetails
  2  VALUES ('BD6', 'B6', 107, 1, 30000);

1 row created.

SQL>
SQL> INSERT INTO BookingDetails
  2  VALUES ('BD7', 'B7', 108, 1, 45000);

1 row created.
```

SELECT BOTH TABLES:

```
SQL> SELECT * FROM Booking;

BOOKING_ID CUSTOMER_ID BOOKING_D TOTAL_AMOUNT BOOKING_STATUS
---------- ----------- --------- ------------ --------------------
B1                  301 10-SEP-26        24000 Confirmed
B2                  302 11-SEP-26        36000 Confirmed
B3                  303 12-SEP-26        55000 Processing
B4                  304 13-SEP-26        65000 Confirmed
B5                  305 14-SEP-26        25000 Completed
B6                  306 15-SEP-26        30000 Processing
B7                  307 16-SEP-26        45000 Completed

7 rows selected.
```

```
SQL> SELECT * FROM BookingDetails;

BOOKING_DE BOOKING_ID PACKAGE_ID NUMBER_OF_TRAVELLERS      PRICE
---------- ---------- ---------- -------------------- ----------
BD1        B1                101                    2      12000
BD2        B2                102                    2      18000
BD3        B3                103                    1      55000
BD4        B4                104                    1      65000
BD5        B5                105                    1      25000
BD6        B6                107                    1      30000
BD7        B7                108                    1      45000

7 rows selected.
```

Customer Booking History

The Customer Booking History Report displays the booking information associated with each customer.

It displays:

- Customer Name
- Booking ID
- Booking Date
- Total Amount
- Booking Status

The report uses a JOIN between Customer and Booking using Customer_ID.

Purpose:

This report helps identify the bookings made by each customer along with the booking date, amount, and current status.

This is the Travel Booking equivalent of the Customer Order History report in the source.

```
SQL> SELECT c.Customer_Name,
  2         b.Booking_ID,
  3         b.Booking_Date,
  4         b.Total_Amount,
  5         b.Booking_Status
  6  FROM Customer c
  7  JOIN Booking b
  8  ON c.Customer_ID = b.Customer_ID
  9  ORDER BY c.Customer_Name;

CUSTOMER_NAME
--------------------------------------------------------------------------------
BOOKING_ID BOOKING_D TOTAL_AMOUNT BOOKING_STATUS
---------- --------- ------------ --------------------
Anu
B2         11-SEP-26        36000 Confirmed

Divya
B4         13-SEP-26        65000 Confirmed

Harini
B6         15-SEP-26        30000 Processing


CUSTOMER_NAME
--------------------------------------------------------------------------------
BOOKING_ID BOOKING_D TOTAL_AMOUNT BOOKING_STATUS
---------- --------- ------------ --------------------
Keerthana
B7         16-SEP-26        45000 Completed

Palin
B1         10-SEP-26        24000 Confirmed

Priya
B3         12-SEP-26        55000 Processing


CUSTOMER_NAME
--------------------------------------------------------------------------------
BOOKING_ID BOOKING_D TOTAL_AMOUNT BOOKING_STATUS
---------- --------- ------------ --------------------
Varshini
B5         14-SEP-26        25000 Completed


7 rows selected.
```

Travel Package-wise Booking Report

The Travel Package-wise Booking Report provides information about the booking frequency and number of travellers for each travel package.

It finds:

- Number of times each package was booked.
- Total number of travellers for each package.

The report joins TravelPackage and BookingDetails using Package_ID.

Functions used:

COUNT()
Counts the number of times a package was booked.

SUM()
Calculates the total number of travellers.

Purpose:

This report helps the travel booking system analyze which packages are being booked and how many travellers selected each package.

This corresponds to the source's artwork-wise order report.

```
SQL> SELECT p.Package_Name,
  2         COUNT(bd.BOOKING_DETAILS_ID) AS Times_Booked,
  3         SUM(bd.Number_of_Travellers) AS Total_Travellers
  4  FROM TravelPackage p
  5  JOIN BookingDetails bd
  6  ON p.Package_ID = bd.Package_ID
  7  GROUP BY p.Package_Name
  8  ORDER BY p.Package_Name;

PACKAGE_NAME
--------------------------------------------------------------------------------
TIMES_BOOKED TOTAL_TRAVELLERS
------------ ----------------
Dubai Holiday Package
           1                1

Goa Honeymoon Package
           1                1

Kashmir Honeymoon Package
           1                1


PACKAGE_NAME
--------------------------------------------------------------------------------
TIMES_BOOKED TOTAL_TRAVELLERS
------------ ----------------
Kerala Family Tour
           1                2

Manali Adventure Trip
           1                1

Ooty Holiday Package
           1                2


PACKAGE_NAME
--------------------------------------------------------------------------------
TIMES_BOOKED TOTAL_TRAVELLERS
------------ ----------------
Singapore Tour Package
           1                1


7 rows selected.
```

Customer Booking Analysis

The Customer Booking Analysis Report analyzes the booking activity of customers.

It finds:

- Total number of bookings by each customer.
- Total spending by each customer.
- Average booking value.

Functions used:

COUNT()
Counts the total number of bookings.

SUM()
Calculates the total amount spent.

AVG()
Calculates the average booking amount.

The report joins Customer and Booking using Customer_ID.

Purpose:

This report provides a summarized view of customer booking activity.

```
SQL> SELECT c.Customer_Name,
  2         COUNT(b.Booking_ID) AS Total_Bookings,
  3         SUM(b.Total_Amount) AS Total_Spending,
  4         AVG(b.Total_Amount) AS Average_Booking_Value
  5  FROM Customer c
  6  JOIN Booking b
  7  ON c.Customer_ID = b.Customer_ID
  8  GROUP BY c.Customer_Name
  9  ORDER BY Total_Bookings DESC;

CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_BOOKINGS TOTAL_SPENDING AVERAGE_BOOKING_VALUE
-------------- -------------- ---------------------
Varshini
             1          25000                 25000

Anu
             1          36000                 36000

Keerthana
             1          45000                 45000


CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_BOOKINGS TOTAL_SPENDING AVERAGE_BOOKING_VALUE
-------------- -------------- ---------------------
Harini
             1          30000                 30000

Divya
             1          65000                 65000

Palin
             1          24000                 24000


CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_BOOKINGS TOTAL_SPENDING AVERAGE_BOOKING_VALUE
-------------- -------------- ---------------------
Priya
             1          55000                 55000


7 rows selected.
```

CRUD Operations

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations used to manage data in a database. In the Travel Booking Management System, CRUD operations are performed on the BookingDetails table to add, view, modify, and remove booking information.

Update Operation

The Update Operation is used to modify an existing record in the BookingDetails table. In the Travel Booking Management System, the number of travellers can be changed when a customer modifies their booking.

```
SQL> UPDATE BookingDetails
  2  SET Number_of_Travellers = 3
  3  WHERE BOOKING_DETAILS_ID = 'BD1';

1 row updated.

SQL> SELECT * FROM BookingDetails
  2  WHERE BOOKING_DETAILS_ID = 'BD1';

BOOKING_DE BOOKING_ID PACKAGE_ID NUMBER_OF_TRAVELLERS      PRICE
---------- ---------- ---------- -------------------- ----------
BD1        B1                101                    3      12000
```

Delete Operation

The Delete Operation is used to remove an existing record from the BookingDetails table. It can be used when a booking detail is no longer required.

```
SQL> DELETE FROM BookingDetails
  2  WHERE BOOKING_DETAILS_ID = 'BD7';

1 row deleted.

SQL> SELECT * FROM BookingDetails;

BOOKING_DE BOOKING_ID PACKAGE_ID NUMBER_OF_TRAVELLERS      PRICE
---------- ---------- ---------- -------------------- ----------
BD1        B1                101                    3      12000
BD2        B2                102                    2      18000
BD3        B3                103                    1      55000
BD4        B4                104                    1      65000
BD5        B5                105                    1      25000
BD6        B6                107                    1      30000

6 rows selected.
```

Conclusion

The Travel Booking Management System was successfully implemented using the concepts covered in Week 4. The Booking and BookingDetails tables were created with appropriate primary keys, foreign keys, NOT NULL constraints, and CHECK constraints.

Relationships were established between customers, bookings, booking details, and travel packages. Sample booking data was inserted and SQL joins were used to generate customer booking history, package-wise booking analysis, and customer booking analysis reports.